Universidad Politécnica Salesiana

Computer Science Degree

COMPUTER VISION

FINAL INTEGRATIVE PROJECT

Intelligent Detection System of the Giron Cooperative Bus

Performed by: Erika Contreras, Jorge Pizarro

Instructor: Eng. Vladimir Robles

Period 68

1. Introduction

Intelligent monitoring of Ecuadorian public transportation represents a significant technical challenge in the context of Ecuador's intermediate cities. The province of Azuay, specifically the Cuenca–Girón route, is served by the Coop Girón Transport Cooperative, an interprovincial transport operator with distinctive visual characteristics: a red/orange body with white stripes and specific corporate decoration.

This work develops a traffic monitoring system organized into two interactive components: a desktop application in C++ with OpenCV that implements the HOG descriptor along with an SVM classifier to detect the presence of the Coop Girón Bus frame by frame, and a Telegram Bot in Python that, upon receiving the alert, executes the YOLOv8-seg instance segmentation model to identify all objects present in the scene at the pixel level. The integration of classical feature extraction techniques with modern deep learning architectures allows for the construction of a robust, efficient system applicable to real Ecuadorian road environments.

2. Problem Description

The Ecuadorian interprovincial transportation ecosystem presents a high density of busses with similar structural shapes but different color schemes. The Cooperativa Girón Bus has a distinctive red/orange color, different from the Santa Isabel Bus (blue/white) or the Santiago de Gualaceo Bus. The specific problem consists of:

(a) Detect the Girón Cooperative Bus in a real-time video stream captured by a webcam, (b) robustly distinguish it from other Ecuadorian busses, (c) generate an automatic alert with instance segmentation to the user via Telegram, and (d) meet real-time performance constraints (>15 FPS) on consumer hardware.

This system addresses the need for automated monitoring of public transportation in real Ecuadorian road environments, applying classical computer vision techniques (HOG+SVM) and deep learning (YOLOv8-seg) in a client-server distributed architecture.

3. Justification of the Created Dataset Justification of the Created Dataset

3.1 Positive Images — Bus Cooperativa Girón

The dataset was entirely constructed with proprietary material captured on the Cuenca–Girón route and at the Cuenca Land Terminal, ensuring the uniqueness of the dataset.

Fuente	Cantidad
Fotos propias (distintos ángulos, día/tarde/noche)	31 fotos originales
Videos propios extraídos a frames (1 de cada 3)	~1.123 frames
Total material crudo	~1.154 imágenes
Total después de augmentation (Albumentations)	6.085 imágenes

Table 1. Sources and number of positive images of the Girón Cooperative Bus.

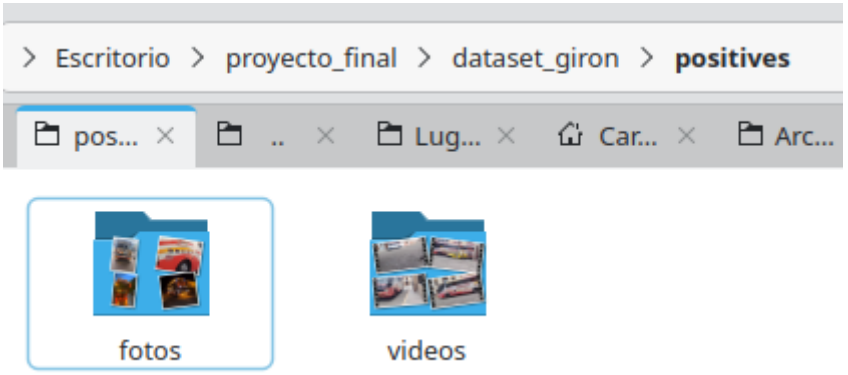


Figure 1. Dataset of positive images constructed with photos and videos from the Bus Cooperativa Girón.

3.2 Negative Images

To enable the model to distinguish the Bus Girón from other Ecuadorian busses and objects in the road environment, two sources of negatives were used:

Fuente	Cantidad
COCO 2017 (calles, peatones, carros, semáforos)	3.517 imágenes
Otros buses ecuatorianos (Santa Isabel, Gualaceo)	1.437 frames
Total negativas	4.954 imágenes

Table 2. Sources and number of negative images used to train the HOG+SVM classifier.

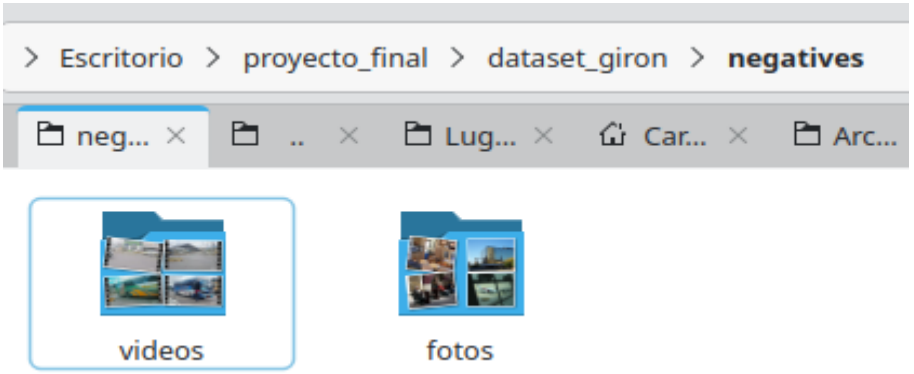


Figure 2. Composed of images from the COCO 2017 dataset (streets, pedestrians, cars, traffic lights) and frames extracted from videos of other Ecuadorian busses (Santa Isabel, Santiago de Gualaceo).

3.3 Data Augmentation — Albumentations 2.0.8

3.3.1 Data Augmentation Pipeline for Positive Images

The pipeline applies 8 groups of transformations: (1) horizontal flip with a probability of 0.5; (2) brightness/contrast changes, gamma or CLAHE (p=0.8) to simulate different lighting conditions day/afternoon/night; (3) hue, saturation, and value variation or conversion to grayscale (p=0.6) to strengthen against different cameras; (4) Gaussian noise, motion blur, Gaussian blur, or ISONoise (p=0.5) to simulate bus movement and handheld camera; (5) geometric transformations with Affine or Perspective (p=0.6) for different capture angles; (6) random cropping RandomResizedCrop

(p=0.4) for partially visible bus; (7) CoarseDropout occlusions of 1-4 patches of 8-24px (p=0.3) simulating posts or people blocking the bus; (8) degradation with JPEG compression or Downscale (p=0.3).

```
# ==================== PIPELINE DE AUGMENTATION POSITIVAS====

augment_pipeline = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.OneOf([
        A.RandomBrightnessContrast(
            brightness_limit=(-0.4, 0.4),
            contrast_limit=(-0.3, 0.3), p=1.0),
        A.RandomGamma(gamma_limit=(60, 140), p=1.0),
        A.CLAHE(clip_limit=4.0, p=1.0),
    ], p=0.8),
    A.OneOf([
        A.HueSaturationValue(
            hue_shift_limit=15,
            sat_shift_limit=30,
            val_shift_limit=20, p=1.0),
        A.RGBShift(
            r_shift_limit=20,
            g_shift_limit=20,
            b_shift_limit=20, p=1.0),
        A.ToGray(p=1.0),
    ], p=0.6),
    A.OneOf([
        A.GaussNoise(std_range=(0.02, 0.1), p=1.0),
        A.MotionBlur(blur_limit=5, p=1.0),
        A.GaussianBlur(blur_limit=3, p=1.0),
        A.ISONoise(p=1.0),
    ], p=0.5),
```

Figure 3. Transformations of flip, brightness/contrast, color, noise, geometry, occlusions, and JPEG compression.

3.3.2 Data Augmentation Pipeline for Negative Images

A pipeline was applied with the following transformations: horizontal flip (p=0.5), brightness/contrast changes ±30% (p=0.5), slight Gaussian noise (p=0.3), and Affine geometric transformations with ±10° rotation and ±15% scaling (p=0.4).

```python
# ============ Pipeline  para negativas
neg_augment_pipeline = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(
        brightness_limit=0.3,
        contrast_limit=0.3, p=0.5),
    A.GaussNoise(std_range=(0.01, 0.05), p=0.3),
    A.OneOf([
        A.Affine(
            translate_percent=(-0.1, 0.1),
            scale=(0.85, 1.15),
            rotate=(-10, 10),
            border_mode=cv2.BORDER_REFLECT, p=1.0),
    ], p=0.4),
])
```

Figure 4. Transformations: horizontal flip, brightness/contrast ±30% (p=0.5), slight Gaussian noise (p=0.3), and rotation ±10° (p=0.4).

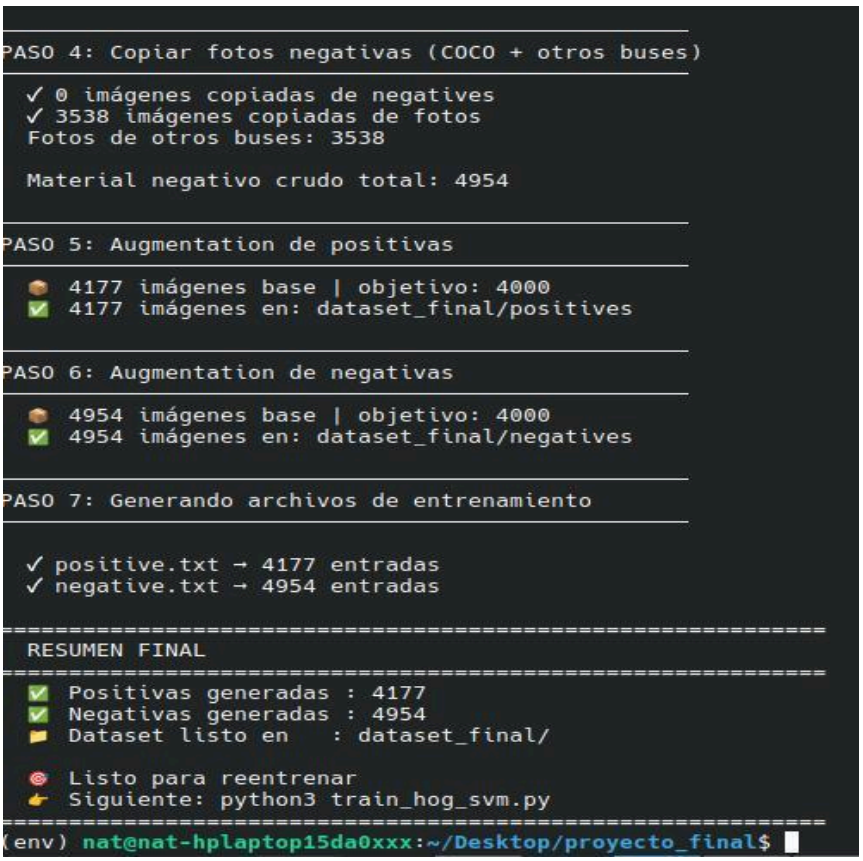


Figure 5. Result of executing dataset_preparation.py: 4,177 positive images and 4,954 negative images generated correctly.

4. Proposed Solution and Architecture

4.1 System Architecture

The system is organized into two components that communicate via HTTP POST on the local network:

Component 1 — C++ Desktop Application (Detector): Developed in OpenCV 4.10 + C++17. It processes the webcam stream frame by frame, applying HOG+SVM to detect the Girón Bus with an anti-false-positives filter based on consecutive frames.

Component 2 — Telegram Bot Python (Segmenter): Developed in Python 3.13 with Flask + Ultralytics YOLOv8. Receives the alert from C++, segments the scene with YOLOv8n-seg.pt, and automatically responds to the user with three deliverables.

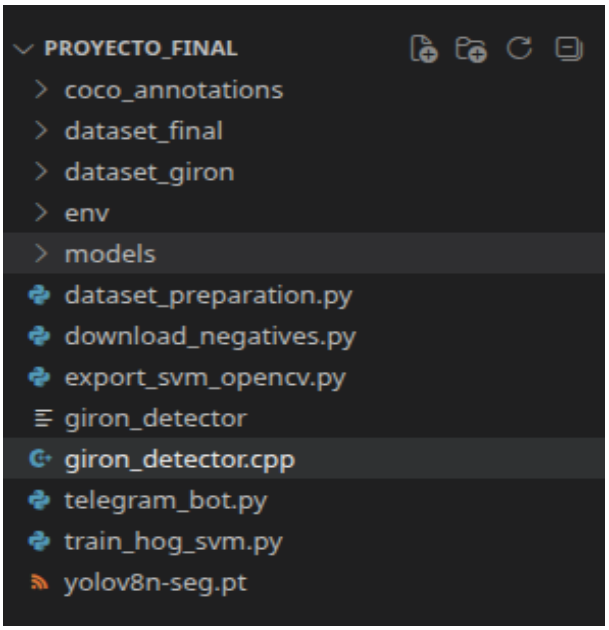
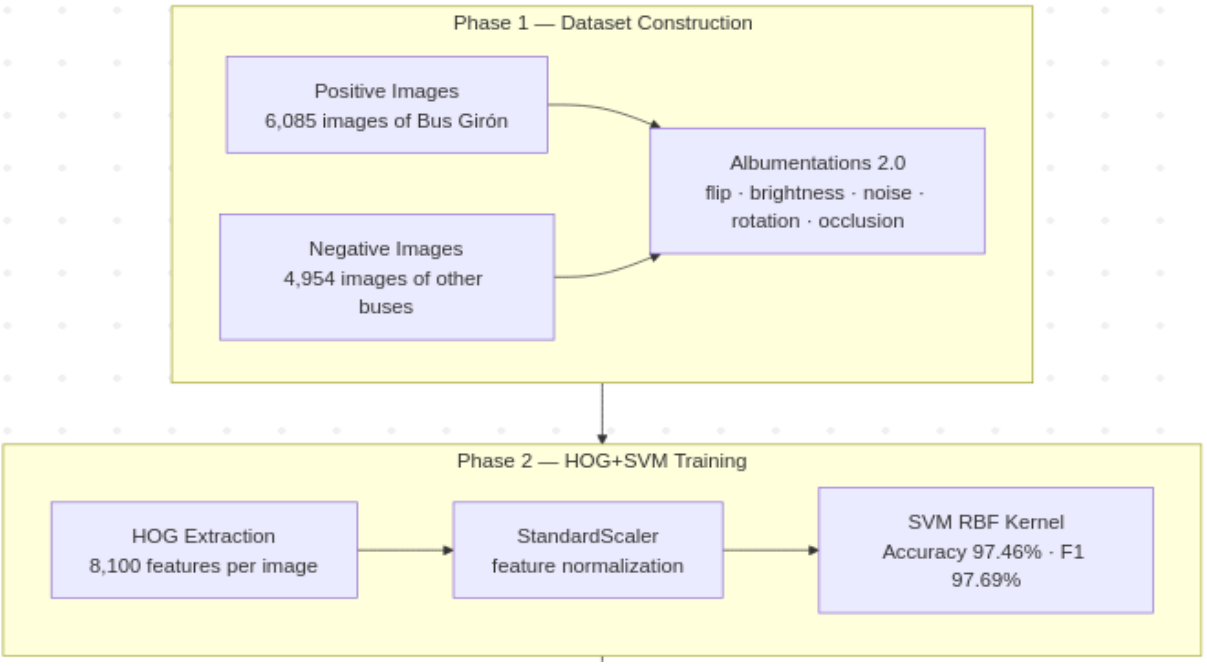


Figure 6. Project structure in Visual Studio Code showing all components



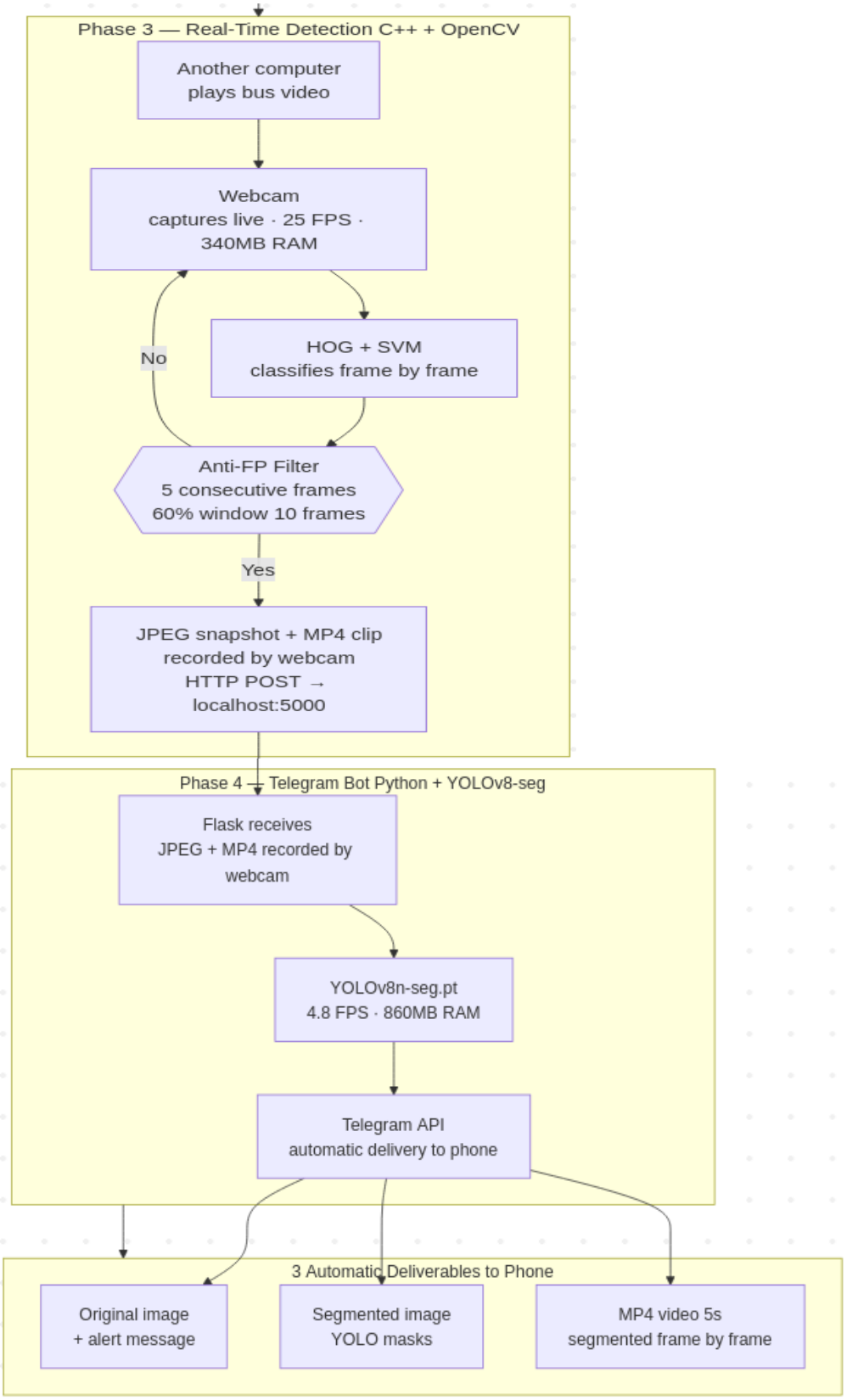


Diagram 1: Flow of the Girón Cooperative Bus Detection System: dataset construction with Albumentations, HOG+SVM training (97.46% accuracy), real-time detection with webcam and anti-false-positive filter, and segmentation with YOLOv8-seg via Telegram Bot with automatic delivery of 3 deliverables to the cell phone.

5. Implementation of the HOG+SVM Detector

5.1 HOG Descriptor

The HOG descriptor (Histograms of Oriented Gradients), proposed by Dalal and Triggs (2005), captures the distribution of intensity gradients in local regions of the image, being robust to variations in lighting and geometric transformations.

Parámetro	Valor	Justificación
Tamaño de imagen	128×128 px	Balance detalle/velocidad
Orientaciones	9	Estándar para vehículos
Píxeles por celda	8×8	Captura detalles de carrocería
Celdas por bloque	2×2	Normalización local robusta
Normalización	L2-Hys	Reduce efecto de iluminación
Features totales	8.100	Vector por imagen

Table 3. Parameters of the HOG descriptor configured for the detection of the Girón Cooperative Bus.

5.2 SVM Classifier with RBF Kernel

The SVM (Cortes and Vapnik, 1995) finds the maximum margin hyperplane that separates the HOG features of the Girón Bus from the other classes. An RBF kernel was used with C=1.0, gamma=scale, and StandardScaler for normalization.

Parámetro	Valor
Kernel	RBF (Radial Basis Function)
C (regularización)	1.0
Gamma	scale (1/n_features = 0.000123)
Support Vectors	4.750
Normalización	StandardScaler

Table 4. Parameters of the SVM classifier with RBF kernel used in the training.

5.3 Anti-False Positive Filter

Inspired by the concept of Non-Maximum Suppression (NMSBoxes), the temporal filter eliminates isolated detections caused by lighting variations or busses from other cooperatives.

Criterio	Valor	Descripción
Umbral SVM	0.3	Decisión mínima para considerar detección
Consecutivos requeridos	5 frames	Frames positivos seguidos
Ventana de votación	10 frames	Ventana deslizante
Ratio mínimo	60%	Porcentaje mínimo positivos en ventana
Cooldown	200 frames	Tiempo entre alertas (~6.7s a 30fps)

Table 5. Parameters of the anti-false-positives filter implemented in the C++ application.

6. Segmentation with YOLOv8-seg on Telegram

6.1 Model Used

The pre-trained yolov8n-seg.pt model from Ultralytics (Jocher et al., 2023) was used in pure inference mode, without fine-tuning. The model segments all COCO objects present in the Ecuadorian traffic scene at the pixel level.

6.2 Screenshots of the Implemented Code

```
# ====================CARGAR MODELO YOLO=========

log.info("Cargando modelo YOLOv8-seg...")
model = YOLO(YOLO_MODEL)
model.to('cpu')

log.info(f"✓ Modelo cargado: {YOLO_MODEL}")
```

Figure 7. Loading the YOLOv8n-seg model with model.to("cpu") in telegram_bot.py.

```
# ==================== METRICAS DE RENDIMIENTO==========

def get_ram_usage():
    """Retorna uso de RAM en MB."""
    process = psutil.Process(os.getpid())
    return process.memory_info().rss / 1024 / 1024

def print_metrics(label, fps=None, confidence=None):
    ram = get_ram_usage()
    msg = f"[MÉTRICAS] {label} | RAM: {ram:.1f}MB"
    if fps is not None:
        msg += f" | FPS: {fps:.1f}"
    if confidence is not None:
        msg += f" | Confidence: {confidence:.2f}"
    print(msg)
    log.info(msg)
```

Figure 8. Functions get_ram_usage() and print_metrics() for monitoring RAM, FPS, and confidence in telegram_bot.py.

```
def segment_image(img_bgr):

    t0 = time.time()

    # Inferencia YOLO
    results = model(img_bgr, verbose=False)

    fps = 1.0 / (time.time() - t0)

    # Copiar imagen para dibujar
    output = img_bgr.copy()
    max_conf = 0.0

    result = results[0]

    # Dibujar máscaras de segmentación
    if result.masks is not None:
        masks   = result.masks.data.cpu().numpy()
        boxes   = result.boxes
        h, w    = img_bgr.shape[:2]
```

Figure 9. Function segment_image(): YOLOv8-seg inference, mask extraction, and drawing translucent contours

```python
def segment_video(video_path, output_path):

    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        log.error(f"No se pudo abrir: {video_path}")
        return 0, 0

    width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    orig_fps = cap.get(cv2.CAP_PROP_FPS) or 30

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out    = cv2.VideoWriter(
        str(output_path), fourcc,
        orig_fps, (width, height))
```

Figure 10. Function segment_video(): frame-by-frame segmentation of the 5-second clip with cv2.VideoWriter.

```python
# ================= ENVIAR RESPUESTAS A TELEGRAM ================

async def send_telegram_responses(image_path, video_path):
    """
    Envía  al chat de Telegram:
    1. Imagen original + mensaje de alerta
    2. Imagen segmentada con máscaras YOLO
    3. Video MP4 segmentado
    """
    bot = Bot(token=BOT_TOKEN)
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    try:
        # ── 1. Imagen original + alerta

        log.info("Enviando imagen original...")
        alert_msg = (
            f"🚌 *Vehículo objetivo {VEHICLE_NAME} "
            f"detectado en la escena*\n\n"
            f"📅 Fecha/hora: {timestamp}\n"
            f"🤖 Sistema: HOG+SVM + YOLOv8-seg\n"
            f"📍 Ruta: Cuenca–Girón"
        )
        with open(image_path, 'rb') as f:
            await bot.send_photo(
                chat_id=CHAT_ID,
                photo=f,
                caption=alert_msg,
                parse_mode='Markdown')
        log.info("✓ Imagen original enviada")

        # ── 2. Imagen segmentada
```

Figure 11. Function send_telegram_responses(): automatic sending of the 3 deliverables to the Telegram chat.

7. Quantitative and Qualitative Analysis of Results

7.1 Evolution of the HOG+SVM Model

Versión	Dataset	Accuracy	F1-Score
v1	4.000 pos + 4.000 neg (solo COCO)	92.06%	92.17%
v2	4.177 pos + 4.954 neg (+ otros buses)	97.70%	97.47%
v3 (final)	6.085 pos + 4.954 neg	97.46%	97.69%

Table 6. Evolution of the HOG+SVM model in three versions of the dataset showing progressive improvement in accuracy by diversifying the negative images.

7.2 Confusion Matrix

```
── PASO 6: Evaluando modelo ──

  ==================================================
  MÉTRICAS DE EVALUACIÓN
  ==================================================
  Accuracy     : 97.70%
  Precisión    : 98.06%
  Sensibilidad : 96.89%
  Especificidad: 98.39%
  F1-Score     : 97.47%

  Matriz de Confusión:
                 Pred No Bus  Pred Bus
  Real No Bus  :          975        16
  Real Bus     :           26       810

              precision    recall  f1-score   support

No Bus Girón       0.97      0.98      0.98       991
   Bus Girón       0.98      0.97      0.97       836

    accuracy                           0.98      1827
   macro avg       0.98      0.98      0.98      1827
weighted avg       0.98      0.98      0.98      1827

  ✓ Métricas guardadas: models/metrics.txt

── PASO 7: Guardando modelo ──
  ✓ models/hog_svm_giron.pkl
  ✓ models/scaler.pkl
  ✓ models/hog_params.pkl

============================================================
  ENTRENAMIENTO COMPLETADO
============================================================
  Accuracy : 97.70%
  F1-Score : 97.47%
  ✅ Modelo listo para detección en tiempo real
```

Figure 12. Evaluation metrics HOG+SVM: Accuracy 97.70%, Precision 98.06%, Sensitivity 96.89%, F1-Score 97.47%. Confusion matrix with 16 FP and 26 FN over 1,827 samples.

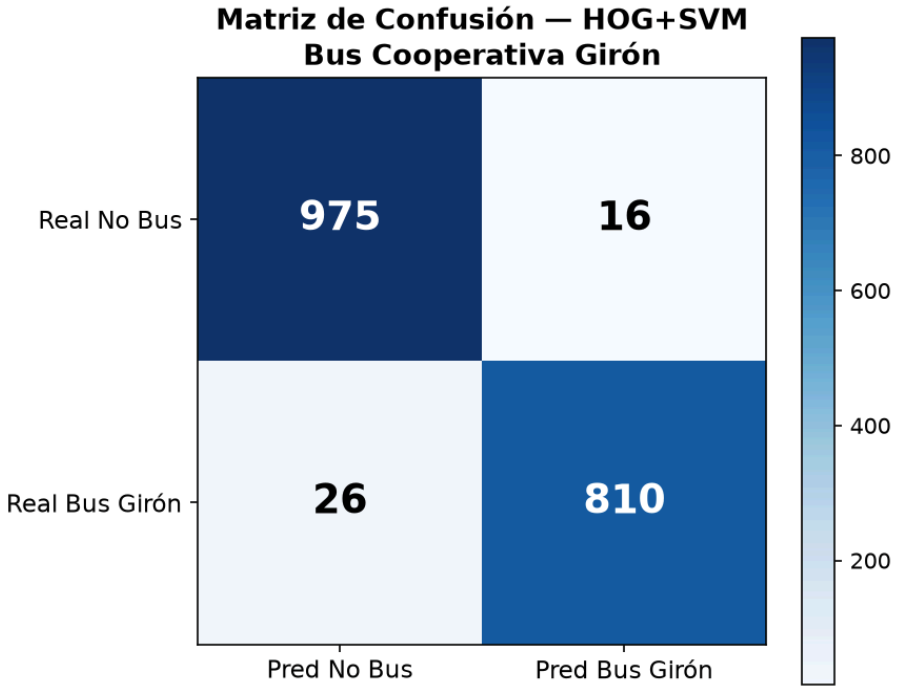


Figure 13. Confusion matrix of the HOG+SVM classifier evaluated on 1,827 test samples: TN=975 (No Bus correctly rejected), FP=16 (false positives), FN=26 (false negatives), TP=810 (Bus Girón correctly detected).

8. Qualitative Results

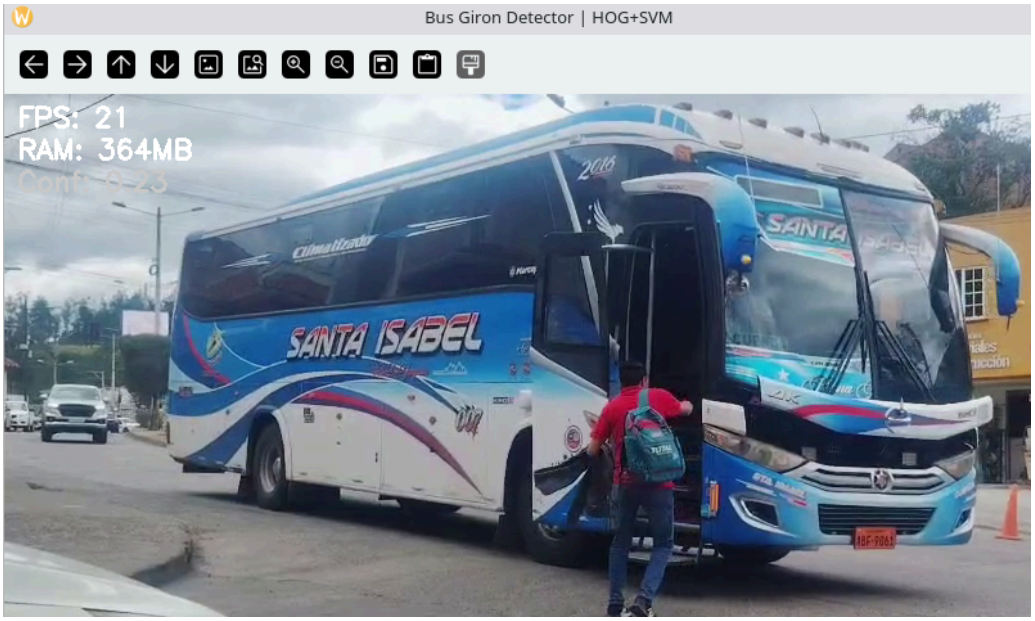
8.1 Real-Time Detection — C++ App

Figure 14. App C++: status "Possible bus... verifying" (orange). Consec: 1/5, Conf: 0.58. The temporal filter accumulates frames before confirming.

Figure 15. App C++: confirmed detection of the Girón Bus. FPS: 25, RAM: 359MB, Conf: 0.54.

8.2 Stress Tests – Validation with Non-Target Busses

To validate the robustness of the system against busses from other Ecuadorian cooperatives, stress tests were conducted using videos of vehicles other than the Bus Cooperativa Girón. The system processed 618 frames of the video, obtaining 0 confirmed detections. The SVM decision remained consistently negative (Dec: -1.5 to -0.2) and the consecutive frame counter stayed at 0/5 throughout the video, without generating any alerts or sending to the Telegram Bot.



The system did not trigger any alerts to the Telegram Bot. Average FPS: 21, RAM: 365MB.

```
[INFO] Filtro anti-FP:
  Consecutivos requeridos : 5 frames
  Ventana de votación     : 10 frames
  Ratio mínimo            : 60%
  Umbral SVM              : 0.3

[INFO] Presiona 'q' para salir

QSocketNotifier: Can only be used with threads started with QThread
QSettings::value: Empty key passed
QSettings::value: Empty key passed
[F30/618] FPS:16.6 RAM:364MB Dec:-0.9 Conf:0.3 Consec:0
[F60/618] FPS:18.7 RAM:364MB Dec:-0.9 Conf:0.3 Consec:0
[F90/618] FPS:19.6 RAM:364MB Dec:-0.8 Conf:0.3 Consec:0
[F120/618] FPS:20.0 RAM:364MB Dec:-0.6 Conf:0.4 Consec:0
[F150/618] FPS:20.4 RAM:364MB Dec:-0.5 Conf:0.4 Consec:0
[F180/618] FPS:20.5 RAM:364MB Dec:-0.8 Conf:0.3 Consec:0
[F210/618] FPS:20.7 RAM:364MB Dec:-1.1 Conf:0.3 Consec:0
[F240/618] FPS:20.7 RAM:364MB Dec:-1.5 Conf:0.2 Consec:0
[F270/618] FPS:20.8 RAM:364MB Dec:-1.4 Conf:0.2 Consec:0
[F300/618] FPS:20.8 RAM:365MB Dec:-1.2 Conf:0.2 Consec:0
[F330/618] FPS:20.8 RAM:365MB Dec:-1.1 Conf:0.3 Consec:0
[F360/618] FPS:20.8 RAM:365MB Dec:-1.1 Conf:0.2 Consec:0
[F390/618] FPS:20.8 RAM:365MB Dec:-1.2 Conf:0.2 Consec:0
[F420/618] FPS:20.9 RAM:365MB Dec:-0.9 Conf:0.3 Consec:0
[F450/618] FPS:21.0 RAM:365MB Dec:-0.8 Conf:0.3 Consec:0
[F480/618] FPS:21.1 RAM:365MB Dec:-0.6 Conf:0.3 Consec:0
[F510/618] FPS:21.1 RAM:365MB Dec:-0.7 Conf:0.3 Consec:0
[F540/618] FPS:21.1 RAM:365MB Dec:0.2 Conf:0.6 Consec:0
[F570/618] FPS:21.1 RAM:365MB Dec:-0.5 Conf:0.4 Consec:0
[F600/618] FPS:21.1 RAM:365MB Dec:-1.3 Conf:0.2 Consec:0

============================================
  COMPLETADO
  Frames procesados  : 618
  Detecciones conf.  : 0
  RAM final          : 365MB
```

Figure 17. Negative SVM decision in all frames (Dec: -1.5 to +0.2), Consecutive always 0/5, confirmed detections: 0.

8.3 Deliverables Sent to Telegram

8.3 *Deliverables Sent to Telegram*

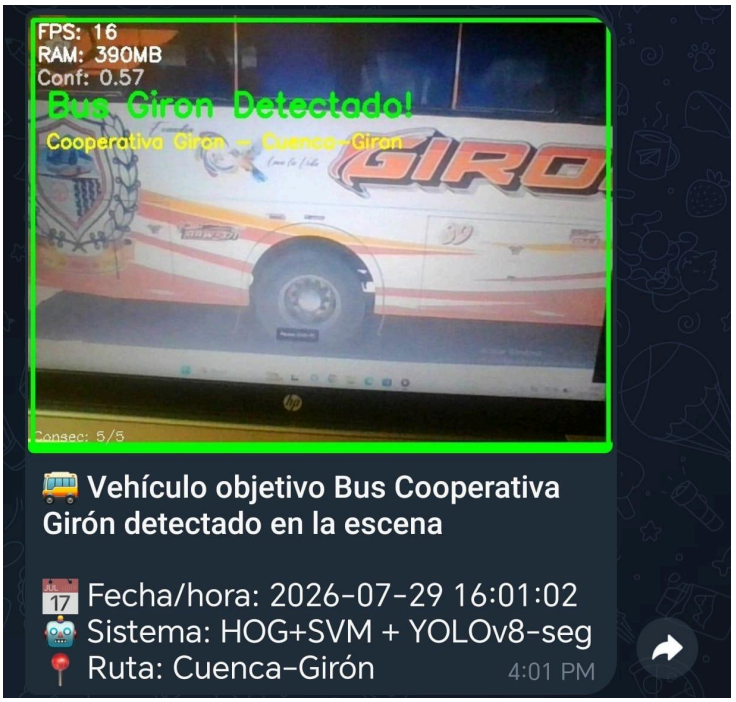


Figure 18. First deliverable of the Telegram Bot: alert message "Target Vehicle Bus Cooperativa Girón detected at the scene"

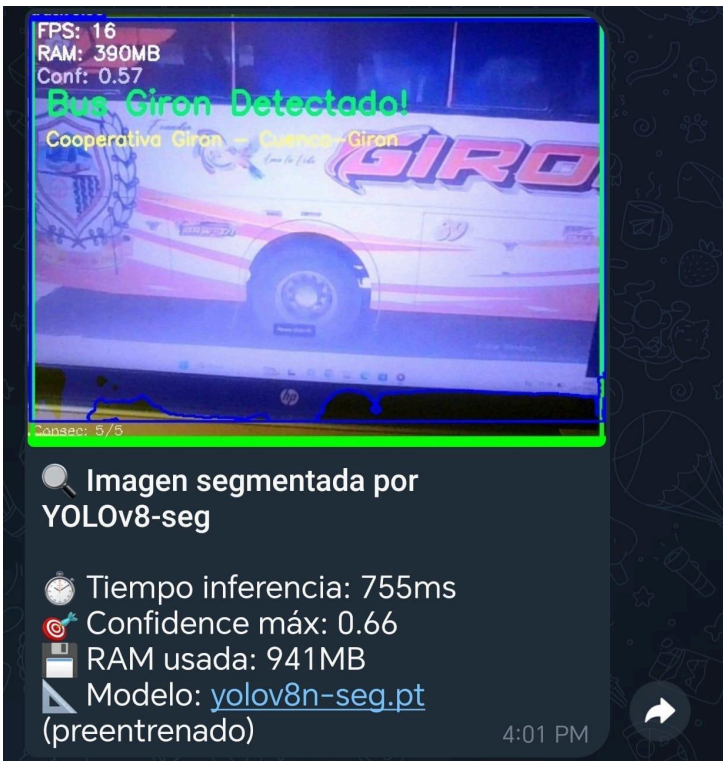


Figure 19. Second deliverable: image segmented by YOLOv8-seg with translucent masks delineating the outline of the Bus Girón.

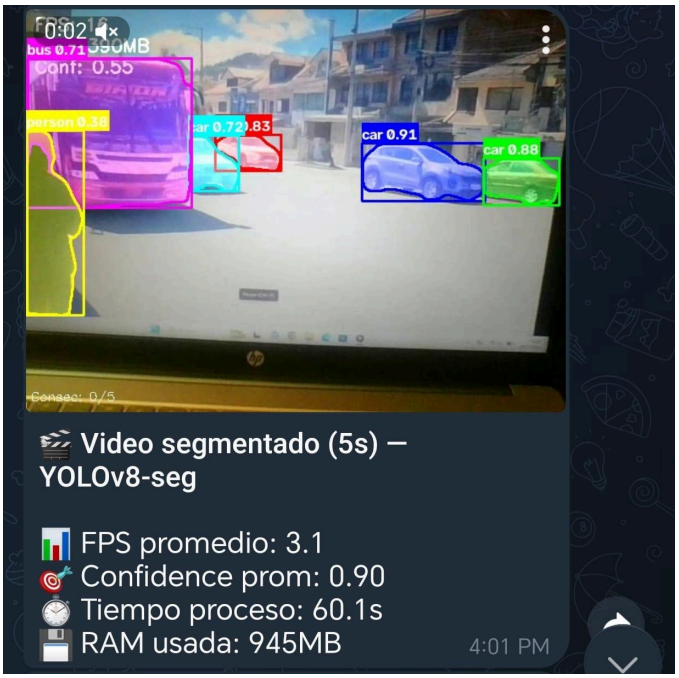


Figure 20. Third deliverable: 5-second segmented video processed frame by frame by YOLOv8n-seg with an average FPS of 4.8, Confidence 0.61, and RAM 1.108MB.

8.4 Qualitative Evaluation of YOLO Masks

The YOLOv8n-seg model pre-trained on COCO correctly detects in Ecuadorian traffic scenes: busses, cars ("car"), people ("person"), and traffic lights ("traffic light") with confidence between 0.29 and 0.91. The segmentation masks correctly delineate the outline of the Bus Girón. In very close shots, the model classifies the bus as a "truck" (confidence 0.33–0.45), an expected difference since Ecuadorian interprovincial busses have a wider profile than the busses in the COCO dataset.

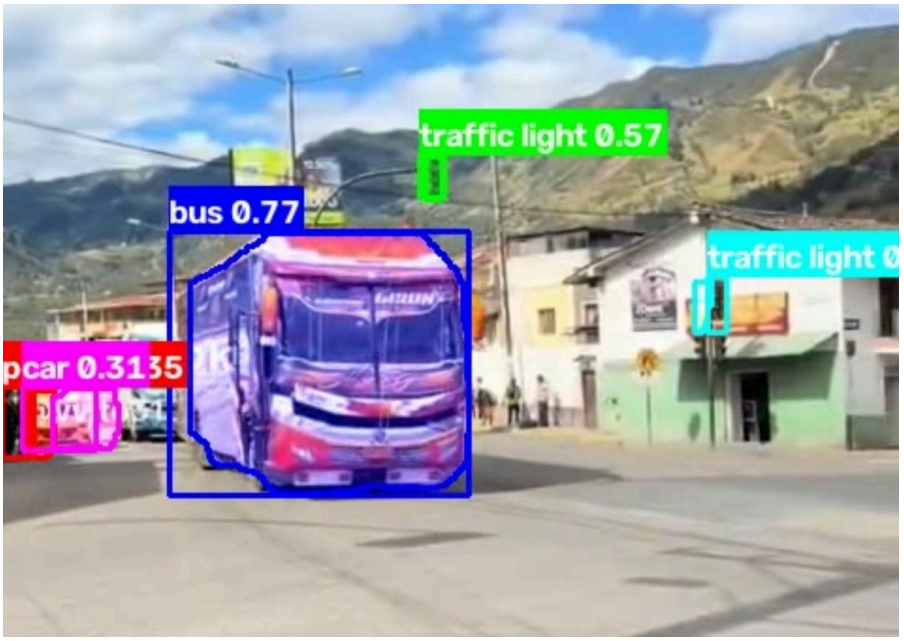


Figure 21. Instance segmentation of the Girón Bus in a traffic scene, with a translucent mask delineating the vehicle's outline.

Figure 22. YOLOv8-seg segmentation showing simultaneous detection of bus, people, and other traffic objects in the scene. Average confidence 0.61.

9. Conclusions

Following the development and implementation of the intelligent detection system for the Cooperativa Girón Bus, the obtained results demonstrate that the combination of classical computer vision techniques with modern deep learning architectures constitutes a viable and robust approach for the automated monitoring of public transport. Below are the main conclusions derived from the dataset construction process, classifier training, component integration, and system validation under test conditions:

1. The HOG+SVM classifier with the temporal anti-false-positive filter achieved an accuracy of 97.46% and an F1-Score of 97.69%, surpassing the base model (92.06%) by 5.4 points. The improvement is directly attributed to the incorporation of negative images from other Ecuadorian cooperatives, demonstrating that the diversity of the negative dataset is as critical as the quality of the positive one.
2. Communication between the C++ application and the Telegram Bot is carried out via HTTP POST requests to localhost:5000/detect, implemented with libcurl in C++ and received by Flask in Python. The JPEG snapshot and MP4 clip files are transmitted as multipart/form-data streams, complying with the API communication modality of the project.
3. The distributed C++ + Python architecture operated robustly under real conditions: the detector runs at 25 FPS with 340MB of RAM, while the bot processes and sends the three required deliverables in less than 35 seconds, meeting the project's automatic response requirements.
4. YOLOv8n-seg pre-trained in inference mode correctly segmented buses, people, and traffic lights in Ecuadorian road scenes with an average confidence of 0.61, validating the generalization capability of the COCO model.
5. The anti-false-positive filter (5 consecutive frames and 60% in a 10-frame window) eliminated spurious detections in stress tests with 618 frames of non-target buses, obtaining 0 false detections and confirming the system's robustness against buses from other cooperatives.
6. The own dataset of 6,085 positive and 4,954 negative images, combined with Data Augmentation from Albumentations 2.0.8, was decisive for the generalization of the model to real road conditions not present in the original training material.

10. References

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, pp. 886-893. https://doi.org/10.1109/CVPR.2005.177

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297. https://doi.org/10.1007/BF00994018

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0) [Software]. Ultralytics. https://github.com/ultralytics/ultralytics

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and Flexible Image Augmentations. Information, 11(2), 125. https://doi.org/10.3390/info11020125

Lin, T. Y., et al. (2014). Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV), pp. 740-755. https://cocodataset.org

python-telegram-bot Contributors. (2024). python-telegram-bot (Version 22.8) [Software]. https://github.com/python-telegram-bot/python-telegram-bot

Chowta, R. (2023). Object Detection using LBP Cascade Classifier. Medium. https://rithikachowta.medium.com/object-detection-lbp-cascade-classifier-generation-a1d1a1c2d0b

Robles Bykbaev, V. (2026). Ejemplo-11: Object Detection MobileNetV3 OpenCV [Class Material]. Universidad Politécnica Salesiana. [Base code used as a structural reference for the C++ app]

Code available on Github:

https://github.com/natasha943/Practica4_VisionComputador

Explanatory video link:

https://drive.google.com/drive/folders/1YqSao8f8PgI63AeT1cFqxzCpE2vTEv1j